

Tight Auditing of Differentially Private Machine Learning

Milad Nasr¹ Jamie Hayes² Thomas Steinke¹ Borja Balle²
Florian Tramèr³ Matthew Jagielski¹ Nicholas Carlini¹ Andreas Terzis¹
¹Google ²DeepMind ³ETHZ

Abstract

Auditing mechanisms for differential privacy use probabilistic means to empirically estimate the privacy level of an algorithm. For **private machine learning, existing auditing mechanisms are tight**: the empirical privacy estimate (nearly) matches the algorithm’s provable privacy guarantee. But these auditing techniques suffer from two limitations. First, **they only give tight estimates under implausible worst-case assumptions** (e.g., a fully adversarial dataset). Second, **they require thousands or millions of training runs to produce non-trivial statistical estimates** of the privacy leakage.

This work addresses both issues. We design an improved auditing scheme that yields tight privacy estimates **for natural (not adversarially crafted) datasets**—if the adversary can see all model updates during training. Prior auditing works rely on the same assumption, which is permitted under the standard differential privacy threat model. This threat model is also applicable, e.g., in federated learning settings. Moreover, our **auditing scheme requires only two training runs** (instead of thousands) to produce tight privacy estimates, by adapting recent advances in tight composition theorems for differential privacy. We demonstrate the utility of our improved auditing schemes by surfacing implementation bugs in private machine learning code that eluded prior auditing techniques.

1 Introduction

Training ML models with stochastic gradient descent (SGD) is not a privacy-preserving function. There is ample evidence that private information from training data can be inferred by observing model parameters trained with SGD or other optimizers [4, 5, 7, 20, 22, 28]. There is also substantial evidence that this privacy risk increases

with the number of model parameters [6, 11], a worrying fact given we are now firmly in the age of large models with hundreds of billions of parameters.

Fortunately, we can train models with differential privacy (DP) guarantees [2, 10], which provably upper bounds any privacy leakage of the training data. Private training typically uses a variant of SGD referred to as Differentially Private Stochastic Gradient Descent (DP-SGD). DP-SGD’s analysis has been conjectured to be overly conservative, and to provide a provable guarantee on privacy leakage that overestimates the leakage in practice [13]. Nasr et al. [21] partially refuted this conjecture by showing that DP-SGD’s analysis gives a tight estimate of the empirical privacy leakage in some worst-case regimes (that fall under the DP threat model). However, their tightness result only holds in a narrow and very strong adversarial model, where the adversary chooses the entire training dataset. This leads to a natural follow-up question:

Q1: Is DP-SGD’s privacy analysis only tight for worst-case datasets?

A further limitation of the approach of Nasr et al.—and other techniques for *auditing* DP-SGD [12, 17, 30]—is the computational overhead. Differential privacy is a probabilistic guarantee, and so empirically estimating an algorithm’s privacy requires computing tight probability estimates of certain events. Existing auditing techniques do this by running the training algorithm thousands of times—which is prohibitively expensive for large models that can cost millions of dollars to train even *once*. Our second question is thus:

Q2: Can DP-SGD’s privacy leakage be tightly estimated with a small number of training runs?

In this work, we design a new auditing scheme for DP-SGD that resolves Q1 and Q2. Our scheme provides

much tighter empirical privacy estimates compared to prior work [12, 17, 21, 30], which match the provable privacy leakage obtained from DP-SGD’s analysis even for non-adversarially-chosen training datasets. We further design methods to reduce the number of models that need to be trained for auditing, from tens of thousands to just **two**. Despite this massive reduction in computational overhead, our empirical privacy estimates remain tight.

Our improvements over prior auditing approaches stem from a fairly simple insight. We observe that existing auditing techniques are *universal*: they make no assumption about the privacy mechanism. We show that “opening the black box” and tailoring our scheme to the specific privacy mechanisms used in DP-SGD results in much tighter empirical privacy estimates and with significantly fewer observations (i.e., training runs). Intuitively, (ϵ, δ) -DP—the standard formulation of DP used in DP-SGD analysis—is concerned with outcomes that have probability $O(\delta)$ and, thus, we need many training runs to observe such outcomes even once. However, we can make inferences about these rare outcomes from common outcomes by leveraging our knowledge about the privacy mechanism. As an analogy, if we know that data follows a Gaussian distribution, then we could estimate the mean and variance from a few samples, and make inferences about the tails of the distribution without ever observing those tails. At a technical level, we adapt existing DP auditing techniques to Gaussian DP and functional DP [9] which provide a more fine-grained characterisation of the privacy leakage for specific mechanisms. We also verify experimentally that our results agree with existing auditing techniques [21, 30] after sufficiently many training runs.

Our improved auditing scheme enables new applications. First, our tight characterization of the empirical privacy leakage of DP-SGD unlocks the ability to directly inspect the impact of various model and training design choices on privacy. We explore how different choices of hyperparameters, model architecture, and the assumed attacker model impact DP-SGD’s empirical privacy leakage.

Second, our tight and computationally efficient auditing enables us to (probabilistically) *verify the correctness* of DP-SGD implementations. Indeed, implementing DP-SGD is notoriously difficult: subtle privacy bugs resulting from incorrect gradient clipping or noising are common and hard to detect [1, 24, 25]. Auditing can help detect such errors by showing that the implementation empirically leaks more information than it should provably allow. However, existing auditing tools are either too expensive to run [21], or provide leakage estimates that

are too loose to catch the most pernicious errors [25, 30]. We show that our improved auditing scheme can surface bugs that would not have been captured by prior methods [12, 17, 21, 30]. We thus encourage developers of differentially private learning algorithms to incorporate our auditing tools into their testing pipeline.

2 Background

We begin with a brief background on differential privacy (DP), private machine learning, and techniques to audit the privacy guarantees claimed under DP.

2.1 Differential privacy

Differential privacy (DP) has become the gold standard method for providing algorithmic privacy [10].

Definition 1 ((ϵ, δ) –Differential Privacy (DP)). *An algorithm $\mathcal{M}$ is said to be (ϵ, δ) -DP if for all sets of events $S \subseteq \text{Range}(\mathcal{M})$ and all neighboring data sets $D, D' \in \mathcal{D}^n$ (where $\mathcal{D}$ is the set of all possible data points) that differ in one sample we have the guarantee:*

$$\Pr[\mathcal{M}(D) \in S] \leq e^\epsilon \Pr[\mathcal{M}(D') \in S] + \delta \quad (1)$$

Informally in the context of machine learning, if a training algorithm $\mathcal{M}$ satisfies (ϵ, δ) -DP then an adversary’s ability to distinguish if $\mathcal{M}$ was run on D or D' is bounded by e^ϵ , and δ is the probability that this upper bound fails to hold.

Trade-off functions and functional DP. There are other useful formalisms of DP. For example, functional differential privacy (f -DP) [9] originates from a hypothesis testing interpretation of differential privacy [14, 26], where an adversary aims to distinguish D from D' given the output of the privacy mechanism. Although our results will be framed using (ϵ, δ) -DP, our auditing framework will operate using functional DP. Consider the following hypothesis testing problem, given some machine learning model f :

H_0 : the model f is drawn from P

H_1 : the model f is drawn from Q

where P and Q are the probability distributions $\mathcal{M}(D)$ and $\mathcal{M}(D')$, respectively. If $\mathcal{M}$ is differentially private, we can derive a bound on an adversary’s power $1 - \beta$ (i.e. True Positive Rate or TPR, where β is the False Negative Rate or Type II error) for this hypothesis test at a significance level α (i.e. False Positive Rate, FPR, or

Type I error). For example, (ϵ, δ) -DP upper-bounds the power of this hypothesis test by $e^\epsilon \alpha + \delta$.

Dong et al. [9] define a *trade-off function* to capture the difficulty in distinguishing the two hypotheses above in terms of the adversary's type I and type II errors. Consider a rejection rule $0 \leq \phi(f) \leq 1$ that takes as input the model f trained by the mechanism $\mathcal{M}$, and which outputs a probability that we should reject the null hypothesis H_0 . This rejection rule has type I error $\alpha_\phi = \mathbb{E}_P[\phi]$ and type II error $\beta_\phi = 1 - \mathbb{E}_Q[\phi]$, which gives rise to the following trade-off function:

Definition 2 (Trade-off function [9]). *For any two probability distributions P and Q on the same space define the trade-off function $T(P, Q) : [0, 1] \rightarrow [0, 1]$ as*

$$T(P, Q)(\alpha) = \inf \{\beta_\phi : \alpha_\phi \leq \alpha\} \quad (2)$$

where the infimum is taken over all rejection rules ϕ .

The trade-off function completely characterizes the boundary of achievable type II errors at a given significance level α , and the optimal test is given by the Neyman-Pearson Lemma. For arbitrary functions f, g defined on $[0, 1]$, we say that $f \geq g$ if $f(\alpha) \geq g(\alpha)$ for all $\alpha \in [0, 1]$. Then, if $T(P, Q) \geq T(\tilde{P}, \tilde{Q})$, this means the distributions P and Q are harder to distinguish than $\tilde{P}$ and $\tilde{Q}$ at any significance level. Thus, a privacy mechanism that produces distributions P and Q on neighboring datasets is strictly more private than one that produces distributions $\tilde{P}$ and $\tilde{Q}$. Dong et al. [9] introduce the following formulation of differential privacy using this insight:

Definition 3 (f -differential privacy (f -DP)). *Let f be a trade-off function. A mechanism $\mathcal{M}$ is f -DP if*

$$T(\mathcal{M}(D), \mathcal{M}(D')) \geq f \quad (3)$$

for all neighboring datasets D and D' .

Dong et al. show that (ϵ, δ) -DP is equivalent to f -DP for the following trade-off function:

$$f_{\epsilon, \delta}(\alpha) = \max\{0, 1 - \delta - e^\epsilon \alpha, e^{-\epsilon}(1 - \delta - \alpha)\} \quad (4)$$

When the underlying distributions P, Q are Gaussian, we get a special case of f -DP called Gaussian DP (GDP) [9]:

Definition 4 (μ -Gaussian Differential Privacy (μ -GDP)). *A mechanism $\mathcal{M}$ is μ -GDP if*

$$T(\mathcal{M}(D), \mathcal{M}(D'))(\alpha) \geq \Phi(\Phi^{-1}(1 - \alpha) - \mu), \forall \alpha \in [0, 1] \quad (5)$$

for all neighboring datasets D and D' , where Φ is the standard normal CDF.

One of the main advantages of GDP is that composition of differential privacy guarantees becomes simple, the composition of two mechanisms following μ_1 -GDP and μ_2 -GDP satisfies μ -GDP with $\mu = \sqrt{\mu_1^2 + \mu_2^2}$. A final fact that will be useful throughout the paper is that it is possible to interpret μ -GDP in terms of (ϵ, δ) -DP:

Corollary 5 (μ -GDP to (ϵ, δ) -DP conversion [9]). *A mechanism is μ -GDP iff it is $(\epsilon, \delta(\epsilon))$ -DP for all $\epsilon \geq 0$, where:*

$$\delta(\epsilon) = \Phi\left(-\frac{\epsilon}{\mu} + \frac{\mu}{2}\right) - e^\epsilon \Phi\left(-\frac{\epsilon}{\mu} - \frac{\mu}{2}\right) \quad (6)$$

2.2 Differentially Private Machine Learning

Stochastic gradient descent (SGD) can be made differentially private through two modifications: clipping individual gradients to maximum Euclidean norm of C and adding random noise to the average of a batch of gradients; this algorithm is commonly referred to as DP-SGD. Intuitively, clipping bounds the individual contribution any sample can make to the model parameters, θ , and adding random noise serves to obfuscate the contributions of any individual example. In practice the update rule for DP-SGD is given as follows: let B denote a batch of examples sampled independently from a dataset D , each with probability q , ℓ be a loss function, and η a learning rate, then

$$\theta \leftarrow \theta - \eta \left(\mathcal{N}(0, \sigma^2 I) + \frac{1}{|B|} \sum_{z \in B} \text{clip}_C(\nabla_{\theta} \ell(\theta, z)) \right) \quad (7)$$

where $\text{clip}_C(v)$ projects v onto the ℓ_2 ball of radius C with

$$\text{clip}_C(v) = v \cdot \min \left\{ 1, \frac{C}{\|v\|_2} \right\}.$$

When we refer to a *privatized gradient*, we mean the gradient after it has been clipped, then averaged, and then noised. To achieve (ϵ, δ) -DP, typically σ is typically on the order of $\Omega(q\sqrt{T \log(1/\delta)}\epsilon^{-1})$ [2] where T is the number of gradient descent iterations, but tighter bounds for a given σ have been found [9, 16, 19]. Each iteration of DP-SGD satisfies a particular (ϵ, δ) -DP guarantee through the subsampled Gaussian Mechanism [2]—a composition of data subsampling and Gaussian noise addition. Since DP is immune to post-processing, we can compose this guarantee over multiple updates to reach a final (ϵ, δ) -DP guarantee.

Unfortunately, a naive composition—by summing the ϵ 's from each iteration—gives values of $\epsilon \gg 10^4$ for accurate neural networks. This yields a trivial upper bound of ≈ 1 on the true positive rate for the hypothesis testing problem discussed in Section 2.1, for any reasonable value of α . Such a large ϵ thus does not guarantee any meaningful privacy. As a result, many works have proposed more sophisticated methods for analyzing the composition of DP-SGD iterations, which can prove much tighter values of $\epsilon < 10$ for *the same algorithm* [2, 9, 16, 19].

2.3 Auditing DP-SGD

Any differentially private algorithm $\mathcal{M}$ bounds an adversary's ability to infer if $\mathcal{M}$ was trained with D or D' . Kairouz *et al.* [14] show that if $\mathcal{M}$ is (ϵ, δ) -DP then it defines a *privacy region* (a bound on an attacker's TPR and FPR) given by

$$\mathcal{R}(\epsilon, \delta) = \{(\alpha, \beta) \mid \alpha + e^\epsilon \beta \geq 1 - \delta \wedge e^\epsilon \alpha + \beta \geq 1 - \delta \wedge \alpha + e^\epsilon \beta \leq e^\epsilon + \delta \wedge e^\epsilon \alpha + \beta \leq e^\epsilon + \delta\} \quad (8)$$

In other words, an (ϵ, δ) -DP algorithm implies a valid region for the type I (α) and type II (β) error of any test.

The goal of a privacy *audit* is to design a hypothesis test that distinguishes D from D' while minimizing α and β . Then, we can compute the privacy budget ϵ , for any fixed value of δ , using Equation (8) (or as we will see later, via other means). In practice, for many interesting differentially private algorithms including DP-SGD, one cannot compute the minimum possible values of α and β in closed form, and so we must rely on empirical estimates. This is done by designing a *distinguisher* that predicts if mechanism $\mathcal{M}$ operated on D or D' . We then run the distinguishing experiment multiple times (i.e., by running $\mathcal{M}$ multiple times to train a model on a random choice of D or D'), collect these observations, and compute empirical lower and upper bounds $\alpha \in (\underline{\alpha}, \bar{\alpha})$ and $\beta \in (\underline{\beta}, \bar{\beta})$ using a binomial proportion confidence interval. Nasr *et al.* [21] use the Clopper-Pearson method to find $\bar{\alpha}$ and $\bar{\beta}$, ultimately deriving an empirical lower bound to ϵ by appealing to Equation (8) and noting that

$$\epsilon_{\text{emp}}^{\text{lower}} = \max \left\{ \ln \left(\frac{1 - \bar{\alpha} - \delta}{\bar{\beta}} \right), \ln \left(\frac{1 - \bar{\beta} - \delta}{\bar{\alpha}} \right), 0 \right\} \quad (9)$$

The lower bound $\epsilon_{\text{emp}}^{\text{lower}}$ comes with an empirical level of confidence through the confidence level for $\bar{\alpha}$ and $\bar{\beta}$. Unfortunately, a high level of confidence in $\epsilon_{\text{emp}}^{\text{lower}}$ often requires thousands or millions of observations.

The adversary is also free to design D and $D' = D \cup \{z\}$ in any way they choose, because the privacy guarantee of DP must hold for *any* pair of neighboring datasets. The goal of the auditor/adversary is thus to design D and z in such a way that it is easy to design a distinguisher for $\mathcal{M}(D)$ and $\mathcal{M}(D')$.

Nasr *et al.* [21] showed that $\epsilon_{\text{emp}}^{\text{lower}}$ is close to the upper bound ϵ output by a DP accounting mechanism when $D = \emptyset$ and so the model is trained on either zero points, or one point z . That is, they designed a test where α and β are minimized under this setting. In summary, this auditing mechanism has shown that current DP accounting methods are nearly tight [2, 9, 16, 19], by showing that the lower bounds for ϵ one can find through a statistical test are close to the upper bound for ϵ given by DP accounting. The drawback of this analysis is that it only demonstrates the analysis is tight with a worst-case dataset, $D = \emptyset$, and to show this it is necessary to train the model thousands of times in order to find non-trivial lower bounds $\epsilon_{\text{emp}}^{\text{lower}}$.

Zanella-Béguelin *et al.* [30] propose a refined Bayesian approach to finding an empirical lower bound for ϵ through a non-informative prior on (α, β) . Specifically, they define a lower bound for ϵ as

$$\underline{\epsilon} = \sup\{\epsilon \in \mathbb{R}_{>0} \mid (\alpha, \beta) \notin \mathcal{R}(\epsilon, \delta)\} \quad (10)$$

From here, they define $f_{(\alpha, \beta)}$ to be the density function of the posterior joint distribution of (α, β) given the observed trials (found through training on $\mathcal{M}(D)$ and $\mathcal{M}(D')$ multiple times). A 100(1- γ)% credible interval $[\underline{\epsilon}, \bar{\epsilon}]$ is then defined as

$$\begin{aligned} \underline{\epsilon} &= \arg \max_{\epsilon} \int \int_{\mathcal{R}(\epsilon, \delta)} f_{(\alpha, \beta)}(x, y) dx dy \leq \frac{\gamma}{2} \\ \bar{\epsilon} &= \arg \min_{\epsilon} \int \int_{\mathcal{R}(\epsilon, \delta)} f_{(\alpha, \beta)}(x, y) dx dy \geq 1 - \frac{\gamma}{2} \end{aligned} \quad (11)$$

There are number of subtle assumptions made in this approach which require unpacking. Equation (11) cannot be evaluated in closed form, and so we must approximate it, and it is not clear how this approximation translates into a statistically sound lower bound. Moreover, the comparison between bounds found through this method and through Clopper-Pearson may be slightly unfair, as they are distinct statements about uncertainty of an estimate. Nevertheless, Zanella-Béguelin *et al.* [30] show that their method dramatically improves the tightness of the lower bound estimate for ϵ in practice. Thus, the number of training runs needed for the audit is also significantly reduced.

In other recent work, Lu *et al.* [17] compute an ϵ lower bound by replacing the Clopper-Pearson method for finding bounds on (α, β) with the Katz-log confidence interval [15], which directly bounds the ratio of binomial

proportions and empirically gives tighter estimates for $\epsilon_{\text{emp}}^{\text{lower}}$ with fewer observations (i.e. the method requires fewer number of models that must be trained on D and D'). However, the Katz-log method gives a confidence bound on the ratio of α to β , and it is not clear if this is valid for (ϵ, δ) -DP where the ratio would change to $\frac{\alpha-\delta}{\beta}$. Lu et al. set $\delta = 0$ in their experiments, giving a lower bound for $(\epsilon, 0)$ -DP. Lu et al. also suggest that empirical privacy leakage is dataset dependent. Our work directly contradicts this claim; we argue that their observations were mostly due to using weaker attacks than are permitted under the DP threat model. By instantiating a more powerful attack our results in Section 6 show that the empirical privacy leakage is close to the theoretical ϵ across a range of datasets.

We note that throughout this work we use the terms *trainer*, *auditor*, and *attacker* interchangeably. The party that *audits* the model takes on the role of an *attacker* to measure the empirical privacy leakage, and this involves *training* the model.

3 Motivation & Threat Model

The goal of our work is to improve the efficiency of empirical privacy estimation. This allows us to study the gap between theoretical and practical privacy bounds. And, as a practical application, auditing methods can be used to validate the correctness of a DP implementation.

Our empirical privacy estimates depend upon the specific threat model we instantiate the test within, and assumptions we place on the adversary. Nasr et al. [21] describe several threat models and settings for auditing machine learning with differential privacy. We similarly study multiple threat models, as there is inevitably a trade-off between the power of the audit (with a powerful adversary) and generalizability of the audit to practical machine learning applications (where the assumptions we make to instantiate a powerful adversary may be unrealistic). We focus on three threat models in decreasing order of attack power.

White-box access with gradient canaries: This is the main threat model considered by the DP-SGD theoretical analysis, and matches the (implicit) threat model assumed by DP. The adversary has access to the privatized gradient and model parameters in every update step and can choose an arbitrary gradient at each update step [4], which we refer to as a *canary* gradient. This canary gradient then gets included into the update with probability q . This mimics an adversary who has access to all aspects of training other than the randomness used

in noise addition and batch selection (i.e., the knowledge of when z was used in training, where $D' = D \cup \{z\}$).

White-box access with input-space canaries: The threat model above assumes the adversary can choose an arbitrary *gradient* that is sampled into a batch of updates. This may be an unrealistic capability for an adversary in practice. Our second threat model removes this assumption, and instead allows adversaries access to intermediate updates, but restricts them to choose an arbitrary training sample (from which gradients are subsequently computed), rather than the ability to choose a gradient directly. We refer to the training sample chosen for each update step as the *canary* sample. This setting matches the threat model of federated learning particularly well, where an adversary can access model updates but may not always have the ability to insert arbitrary gradients into the training pipeline.

Black-box access: One of the most restrictive threat models to conduct audits on is that of an adversary who can only insert a training example at the beginning of training, and observe the model after it has completed training. In other words, the adversary does not get to observe or influence intermediate model updates. While this is the most restrictive setting from an adversarial perspective, it is perhaps the most realistic from a practical standpoint. We stress that this threat model is not the typical setting analyzed in DP, which assumes intermediate model updates are visible to the adversary. We choose to evaluate it because it allows us to compare how the incremental removal of adversarial access to model updates and gradients affects the tightness of our lower bound for ϵ .

4 Auditing with f -DP

To audit the privacy of DP-SGD, an adversary repeatedly runs a distinguishing attack to infer if a model's training set was either D or D' ; by measuring the false positive and false negative rates of the attack we can bound privacy. All prior work has used (ϵ, δ) -DP definition to audit the privacy parameters of the algorithm. The limitation of this approach is that different differential privacy mechanisms with *identical* (ϵ, δ) guarantees can have *different* trade-offs between false positive and false negative rates, which are upper bounded by the trade-off function of (ϵ, δ) -differential privacy. In particular, any (ϵ, δ) -DP guarantee corresponds to two symmetric supporting linear functions defining the trade-off between type I and type II errors. However, any mechanism will

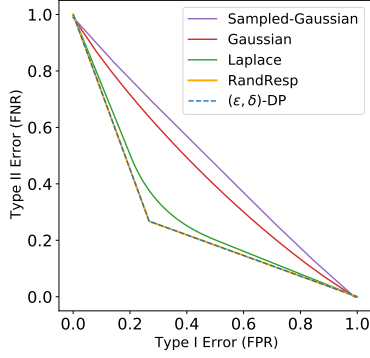


Figure 1: Comparison of the trade-off functions for different DP mechanisms that satisfy (ϵ, δ) -DP where $\epsilon = 1, \delta = 10^{-5}$. Note that the trade-off curve for the Random-Response mechanism overlaps with (ϵ, δ) -DP.

have its own “true” trade-off curve capturing the relationship between the FPR and TPR of the best possible attack on the mechanism, consisting of the intersection between a collection of $(\epsilon, \delta(\epsilon))$ -DP curves where $(\epsilon, \delta(\epsilon))$ satisfy Corollary 5. While it is possible to audit any algorithm to lower bound its privacy with an (ϵ, δ) -DP guarantee, we instead use an f -DP guarantee that is as close as possible to the true trade-off function of the mechanism. By doing so, we can avoid any looseness that appears in converting between this f -DP guarantee and its collection of many (ϵ, δ) -DP guarantees.

To illustrate this idea, in Figure 1 we plot the trade-off functions for several different DP mechanisms that all satisfy (ϵ, δ) -DP where $\epsilon = 1, \delta = 10^{-5}$. Clearly, the achievable false positive and false negative rates by an adversary who wants to audit a (ϵ, δ) -DP guarantee depends significantly on the underlying privacy mechanism.

Let us now give a concrete example demonstrating the benefit of auditing by measuring the privacy region of the private mechanism directly, rather than focusing on the privacy region specified by (ϵ, δ) -DP. Suppose we want to audit an instance of the Gaussian mechanism satisfying $(1, 10^{-5})$ -DP; Figure 2 illustrates the privacy regions of a generic private mechanism with $\epsilon = 1, \delta = 10^{-5}$ and with a Gaussian mechanism which has an equivalent (ϵ, δ) -DP guarantee (i.e., μ -GDP with $\mu \approx 0.25$). Now, if we want to audit the Gaussian mechanism by bounding (ϵ, δ) -DP, our attack needs to have very low false positive or false negative rates (corresponding to the four locations where the GDP region and the (ϵ, δ) -DP region have tangent borders). As a concrete example, suppose we are auditing this Gaussian mechanism (with $\epsilon = 1$) and we have an attack that achieves $\text{FPR} \approx 0.23$ and $\text{FNR} = 1 - \text{FPR}(e^{0.3} + \delta)$ over an infinite number of trials (the

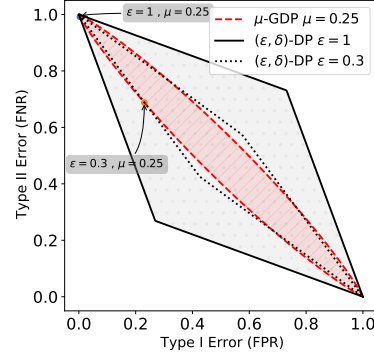


Figure 2: Comparison of the privacy region of (ϵ, δ) -DP vs f -DP for the Gaussian mechanism (GDP) with the same ϵ budget.

red dot in Figure 2). If we use (ϵ, δ) -DP to audit this mechanism we get an empirical ϵ of 0.3, and we might (incorrectly) conclude that our mechanism is not tight. This is not because our attack is weak, but rather it is because our attack has a large FNR and no attack can achieve a lower FNR from the definition of the Gaussian mechanism (i.e., if an attack can achieve a lower FNR it violates ~ 0.25 -GDP).

4.1 Lower Bounding f -DP With Clopper-Pearson

Previous works use Equation (9) which describe the predictive power of an adversary auditing with (ϵ, δ) -DP to compute the privacy parameters from the false positive and negative rates. However, as we have already seen by appealing to the hypothesis testing interpretation of DP, the predictive power of the adversary is by definition equal to the trade-off function of the privacy mechanism. Therefore, instead of using Equation (9) to compute the privacy parameters, we can directly use the trade-off function. Now by upper bounding the false positive (α) and false negative rates (β) (referred to as $\bar{\alpha}, \bar{\beta}$) we can calculate the lower bound on the privacy of the mechanism. Similar to the previous works [21] we can use the Clopper-Pearson method to compute the upper bounds on the attacker errors.

For example, suppose we want to audit the Gaussian mechanism. To compute a lower bound on the privacy parameters of the Gaussian mechanism (i.e., μ), we have:

$$\mu_{\text{emp}}^{\text{lower}} = \Phi^{-1}(1 - \bar{\alpha}) - \Phi^{-1}(\bar{\beta}) \quad (12)$$

We convert this into a lower bound for ϵ by noticing that the lower bound $\mu_{\text{emp}}^{\text{lower}}$ implies an upper bound on the trade-off function of the mechanism at every α . Such an upper bound on the trade-off function enables us to use Equation (4) at a fixed δ , to find the largest lower bound for ϵ over all α .

Improvement: Nasr et al. [21] showed DP-SGD accounting is tight for a worst case dataset ($D = \emptyset$), assuming the adversary has white-box access to all iterations of the training. However, they require many observations to achieve tight bounds, due to the aforementioned drawbacks of auditing with (ϵ, δ) -DP. We re-evaluate this setting using our new approach of auditing with GDP. In Figure 3, we compare the lower bounds found through (ϵ, δ) -DP (Equation (9)) against using the Gaussian trade-off function and converting the Gaussian mechanism parameter to (ϵ, δ) -DP, and we inspect how these two methods compare as the adversary collects more observations from which they compute upper bounds $\tilde{\alpha}$ and $\tilde{\beta}$. While, it is possible for (ϵ, δ) -DP audit to find a lower bound that is tight to the theoretical value for ϵ , this is only achieved when the adversary has 100 million observations. When the number of the observations is smaller there is a non-trivial gap between the theoretical bound and the empirical lower bound. Comparatively, if we use the Gaussian mechanism’s trade-off function (GDP) to estimate a lower bound on the privacy parameter μ , and convert this into a bound on ϵ , we can achieve a tight estimate even with 1,000 observations.

Approximating the trade off function: While using GDP to audit can give us a tight lower bound on the estimated privacy in worst case settings, analyzing the privacy cost of a complex mechanism such as DP-SGD (that needs both sub-sampling amplification and composition over multiple update steps) is non-trivial. Dong et al. [9] suggest to use a Central Limit Theorem for DP to compute a closed form solution, unfortunately, this can lead to significant underestimation of the privacy cost in general settings [16]. Therefore, to analyze DP-SGD we use an empirical approach called the “Privacy Loss Distribution (PLD)” [16] to approximate the trade-off function, which computes a tight privacy cost of DP-SGD over multiple update steps. We refer to Koskela et al. [16] for a detailed description of PLD, and will interact with it as a black-box $\epsilon = f_{\mathcal{M}}(\delta)$ that for private mechanism $\mathcal{M}$ and a given δ will return the exact theoretical ϵ . PLD does not have a closed-form trade-off function, in Appendix A we explain how we use PLD for auditing.

4.2 Lower Bounding f -DP With Bayesian Estimation

Recently, Zanella-Béguelin et al. [30] showed it is also possible to compute *credible intervals* for ϵ using a Bayesian method which can significantly reduce the number of observations to estimate a tight bound. Here, we show it is possible to extend their approach to lower bound in f -DP (and then convert to a lower bound for ϵ).

Definition 6 (Cumulative Distribution Function of $f(\alpha, \cdot)$ -DP). *Let $u_{(FPR, FNR)}$ be the density function of the joint distribution of (FPR, FNR) . The value of cumulative distribution function of $f(\alpha; \cdot)$ evaluated at $f(\alpha; \cdot)$ is:*

$$P(\cdot) = \int_{f(\alpha; \cdot)}^{1-f(1-\alpha; \cdot)} \int_0^1 u_{(FPR, FNR)}(\alpha, \beta) d\alpha d\beta \quad (13)$$

Using Eq. (13), we can find an empirical lower bound for μ in μ -GDP, and then convert to a lower bound for (ϵ, δ) -DP. As shown by Zanella-Béguelin et al. [30], using the CDF of the private mechanism parameters given the attack observations of we can compute credible intervals over ϵ . As a reminder Zanella-Béguelin et al. defined $u_{(FPR, FNR)}$ as follows:

$$\begin{aligned} u_{(FPR, FNR)}(\alpha, \beta) &:= u_{(FPR|FP)}(\alpha) u_{(FNR|FN)}(\beta) \\ &= \text{Beta}(\alpha; 0.5 + FN, 0.5 + N - FN) \times \\ &\quad \text{Beta}(\beta; 0.5 + FP, 0.5 + N - FP) \end{aligned} \quad (14)$$

where N is the number of observations used to compute false positive and false negative rates, and FN , FP are the total number of false negative and false positives, respectively.

5 Auditing Setup

As we have seen, the choice of framework used for auditing can affect the tightness of our lower bound for ϵ . In this section, we describe our auditing procedure for each threat model described in Section 3, and then discuss the effect of different attacker choices—such as attack specific hyperparameters—have on the audit results.

5.1 Auditing Procedure

As mentioned in Section 3, we consider three main threat models. For the black-box setting, we use Algorithm 1 which trains $2T$ models on datasets D and $D' = D \cup \{z\}$ where $z = (x', y')$ is the differing example between

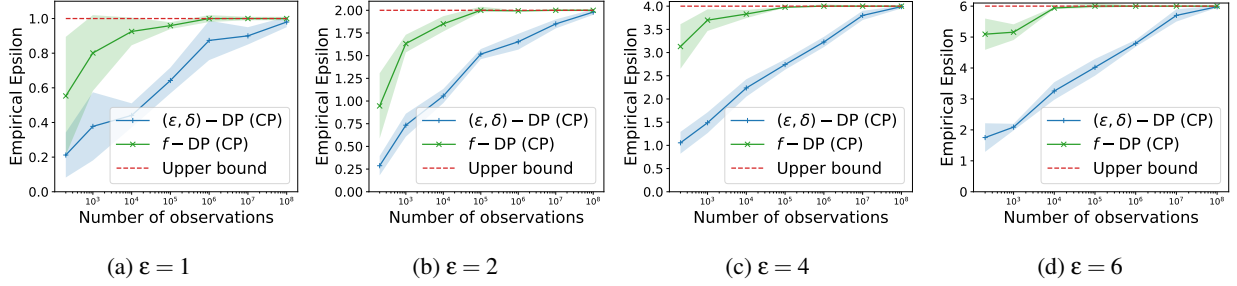


Figure 3: Comparison of the (ϵ, δ) -DP definition to audit DP-SGD compared to using f -DP of Gaussian mechanism both converted in (ϵ, δ) -DP and lower bounded using Clopper Pearson (20 independent runs, $\delta = 10^{-5}$).

D and D' , which we refer to as the *canary*. Then the auditor evaluates the loss on the canary example of each model trained on D and D' ; using this set of losses, the auditor chooses a decision threshold and computes α and β . We refer to the statistics collected by the adversary as *observations*.

Algorithm 1 Black-box auditing for DP-SGD

Args: training dataset D , loss function l , canary input (x', y') , number of observations T
 Observations: $O \leftarrow \{\}, O' \leftarrow \{\}$
for $t \in \{T\}$ **do**
 $\theta \leftarrow$ DP-SGD on Dataset D
 $\theta' \leftarrow$ DP-SGD on Dataset $D \cup (x', y')$
 $O[t] \leftarrow l(\theta, (x', y'))$
 $O'[t] \leftarrow l(\theta', (x', y'))$
end for
return θ, O, O'

In the white-box setting, the adversary can observe model parameters at each update step. We summarize the approach used for auditing in a white-box setting in Algorithm 2, using either canary gradients or canary inputs as described in Section 3. At each iteration of DP-SGD, the trainer independently samples two batches of data, B and B' . In the *White-box access with Input Space Canaries* threat model, the trainer creates a canary sample and adds it to B' with probability q_c .

In the *White-box access with Gradient Canaries* threat model, the trainer computes the batch of per-example gradients for B and B' and adds a canary gradient into B' with probability q_c . Batch B has been sampled from the original dataset D , while B' has been sampled from a modified dataset D' . In both threat models, after the canary is added, DP-SGD proceeds as normal, clipping, aggregating, and noising the gradient sums. For each batch, the trainer computes the dot product between the privatized gradient sum and the canary gradient (or the gradient from the canary input), resulting in a score (which we

again refer to as an observation). The goal of the adversary is to determine whether a batch was drawn from D or D' . At the end of each run of the algorithm, the trainer produces $2T$ observations, two for each update, and a fully trained model with parameters θ .

In Algorithm 2, we consider different sampling rates for the canary example and normal training examples to allow the model trainer to evaluate the mechanism at different sampling rates. If the trainer sets $q_c = 1$, the canary is selected in all iterations, and the audit focuses on the privacy mechanism without data sub-sampling. This modification allows us to identify bugs in DP-SGD that are not due to batch sampling.

After collecting observations from a model trained on either dataset D or D' , they can be compared with a threshold to compute true and false positive rates. Next, we will focus on considerations for choosing an appropriate threshold and their effects on the auditing process. We will also discuss other important factors in the auditing process, such as canary selection strategies and the sampling rate q_c .

5.2 Choosing a Decision Threshold

The output of Algorithm 1 or Algorithm 2 is a set of observations, with canaries $\{o'_1, o'_2, \dots, o'_T\}$, and a set without canaries, $\{o_1, o_2, \dots, o_T\}$. To compute our attack's FNR and FPR, we must first choose a decision threshold to distinguish between observations from the observation space without the canary, O , or the observation space with canaries O' .

In the white-box threat model, our observation is $o = \langle g', \widetilde{\nabla}[t] \rangle$ and $o' = \langle g', \widetilde{\nabla}'[t] \rangle$, where g' is the canary gradient, $\widetilde{\nabla}[t]$ is the privatized gradient over a batch B , and $\widetilde{\nabla}'[t]$ is the privatized gradient over a batch B' . By construction, we expect g' to be orthogonal to any other gradient g in the batch, $\langle g', g \rangle = 0$. In practice, the clip-

Algorithm 2 White-box auditing for DP-SGD with **gradient** or **input** space canaries

Args: training dataset D , sampling rate q , learning rate η , noise scale σ , gradient norm clip C , loss function l , **canary gradient** g' , **canary input** (x', y') , canary sampling rate q_c , function `clip` that clips vectors to max norm C , number of observations T , number of training iterations τ .

Observations: $O \leftarrow \{\}$, $O' \leftarrow \{\}$

Trained Models: $\Theta \leftarrow \{\}$

$t \leftarrow 0$

while $t \leq T$ **do**

 Initiate θ randomly

for τ iterations **do**

$B_t \leftarrow$ sample instances from dataset D with prob q

$B'_t \leftarrow$ sample instances from dataset D with prob q

$\nabla[t] \leftarrow \vec{0}$

for all $(x, y) \in B_t$ **do**

$\nabla[t] \leftarrow \nabla[t] + \text{clip}(\nabla_\theta(l(x, y)))$

end for

$\nabla[t] \leftarrow \nabla[t] + \mathcal{N}(0, \sigma^2 I)$

$\nabla'[t] \leftarrow \vec{0}$

for all $(x, y) \in B'_t$ **do**

$\nabla'[t] \leftarrow \nabla'[t] + \text{clip}(\nabla_\theta(l(x, y)))$

end for

$\nabla'[t] \leftarrow \nabla'[t] + \mathcal{N}(0, \sigma^2 I)$

$\widetilde{g'} \leftarrow$ **canary gradient** or $\widetilde{g'} \leftarrow \text{clip}(\nabla_\theta(l(x', y')))$

$\widetilde{\nabla'[t]} \leftarrow \widetilde{\nabla'[t]} + \widetilde{g'}$ with prob q_c o.w. $\widetilde{\nabla'[t]} + \vec{0}$

$O[t] \leftarrow \langle \widetilde{g'}, \widetilde{\nabla'[t]} \rangle$

$O'[t] \leftarrow \langle \widetilde{g'}, \widetilde{\nabla'[t]} \rangle$

$\theta \leftarrow \theta - \eta \nabla[t]$

$t = t + 1$

end for

$\Theta = \Theta + \{\theta\}$

end while

return Θ, O, O'

ping norm and batch size are known to the adversary and so we can re-scale and normalize the set of observations such that $O = \mathcal{N}(0, \sigma^2 I)$ and $O' = \mathcal{N}(1, \sigma^2 I)$, meaning that in expectation o and o' are sampled from Gaussians with zero and unit mean, respectively. Thus, the attacker's goal is to distinguish observations sampled from $\mathcal{N}(0, \sigma^2 I)$ and $\mathcal{N}(1, \sigma^2 I)$ – we note that this is exactly the same hypothesis testing problem considered in GDP. A benefit of auditing with $q_c = 1$ and GDP is that the lower bounds we derive with GDP are agnostic to our choice of decision threshold used to compute type I and type II errors (in the limit of number of observations), while this is not true for (ϵ, δ) -DP. This is due to the perfect match between the GDP analysis and the true privacy of the Gaussian mechanism. Figure 4 shows the result that different choices of threshold have on auditing using either GDP or (ϵ, δ) -DP. As we can see, auditing using (ϵ, δ) -DP is much more sensitive to the decision threshold compared to GDP. Note that if thresholds we use are found using the same observation data that we

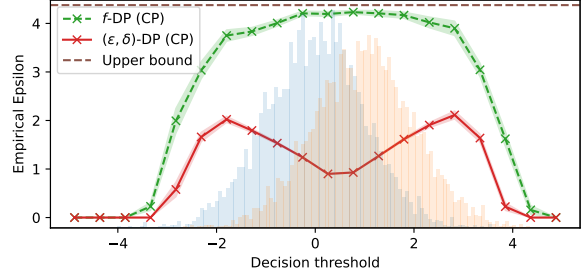


Figure 4: An example comparison between f -DP and (ϵ, δ) -DP with Clopper-Pearson lower bounds for 5,000 observations. The lower bound found through GDP are approximately the same for any decision threshold within the observation's support, whereas the (ϵ, δ) -DP with Clopper-Pearson lower bound varies dramatically. We also visualize observations when the canary was and wasn't included in training as histograms.

compute the lower bound on, the bound is technically not valid. However, it has become common to report lower bounds on the same set of observations that one uses to find an optimal decision threshold [18, 30], and so for each method we will find the threshold that maximizes the reported lower bound. We stress that for GDP, any decision threshold will be equally likely to maximize the lower bound with a sufficient number of observations; this is not true of (ϵ, δ) -DP. We discuss this further in Appendix C.1.

Next, we discuss how to construct the canary point (either gradient or sample) used in auditing.

5.3 Canary Type

The strength of our bound depends on being able to distinguish samples from O and O' . In turn, this means crafting canaries that maximize distinguishability.

In the *White-box access with Gradient Canaries* threat model, we use what we refer to as a Dirac canary gradient; a gradient with zeros everywhere except at a single index in the gradient vector, where we set its value to the clipping norm C . We compare this choice with other possibilities in Appendix C.2.

In the *White-box access with Input Space Canaries* threat model, we design a new attack that crafts an input for given model parameters. We evaluate four different canary strategies: (1) a random sample from the dataset distribution with a wrong label, (2) using a blank sample, (3) an adversarial example, and (4) our new canary crafting approach given in Algorithm 3 and discussed in Appendix C.3. In the *black-box* threat model, we con-

Algorithm 3 Input canary generation in white-box setting

Args: In-distribution dataset D , model loss function l , model parameters θ , T crafting steps, η step size

$$\bar{g}_{dist} = \frac{1}{|D|} \sum_{(x_i, y_i) \in D} \nabla l(\theta, (x_i, y_i))$$

$$l_{adv}(x, y) = \left| \frac{\nabla l(\theta, (x, y)) \cdot \bar{g}_{dist}}{\|\nabla l(\theta, (x, y))\| \|\bar{g}_{dist}\|} \right|$$

$(x, y) \xleftarrow{\$} D$
for $t \in \{T\}$ **do**
 $x = x - \eta \nabla l_{adv}(x, y)$
end for
return (x, y)

sider a similar range of canary types, which are detailed in Section 6.4.

5.4 Canary Sampling Rate (q_c)

The analysis of the sub-sampling mechanism in the worst case is tight using PLD/ f -DP [16]. From Figure 12, we see that when $q_c = 1$, the observations closely match the theoretical FPR-FNR trade-off. Instead, in Figure 5 we audit a sub-sampled Gaussian mechanism with sampling rate of $q_c = \frac{1}{4}$, setting $\sigma^2 = 0.3$ and the number of collected observations to 10,000. This figure plots the FPR-FNR curve predicted by using the PLD accounting and also GDP accounting with an equivalent ϵ with $\delta = 10^{-5}$, and compares it to the empirical curve found through auditing this sub-sampled Gaussian mechanism. Clearly, both PLD and GDP upper bound the observed FPR-FNR curve, but tends to overestimate the trade-off between FPR and FNR, particularly at higher false positive rates [9]. This suggests that to accurately audit a sub-sampled privacy-preserving mechanism, it may be necessary to use attacks with more precise false positive rates and optimal thresholds in order to achieve tight bounds. As mentioned in Section 5.2 (and expanded upon in Appendix C.1) by using $q_c = 1$ (and a sufficient number of observations) we do not need to find the optimal threshold, as the lower bound found using GDP auditing is threshold agnostic, and any threshold will results in tight auditing (which ensures a valid confidence interval and lower bound).

When using auditing to debug an implementation of DP-SGD, we focus on the auditing of the privacy mechanism itself, rather than the sub-sampling process. We therefore set $q_c = 1$ throughout most of the experiments in Section 6. However, we will experiment with the sub-sampled Gaussian mechanism ($q_c < 1$) in the black-box threat model, where we are more focused on effect of the threat model on privacy leakage rather than debugging to check if an implementation of DP-SGD is correct.

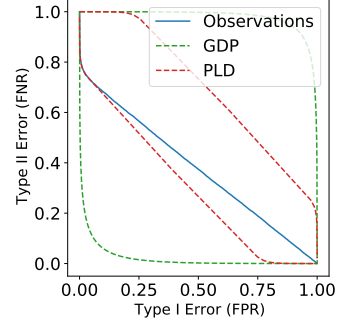


Figure 5: PLD or GDP analysis does not describe the achievable error rate from a sub-sampled Gaussian mechanism.

5.5 From Step-wise to End-to-end Auditing

Our white-box audit observes each step of DP-SGD. As such, we generate privacy lower bounds for individual steps. We convert these into a privacy lower bound for the end-to-end training procedure by appealing to tight composition results. For instance, if we assume that each step is tightly characterized by Gaussian DP, then the tightness of Gaussian DP composition allows us to infer that the end-to-end training procedure is also tightly characterized by Gaussian DP and that we can sum up the privacy parameters. In other words, if the privacy loss distribution of one step is Gaussian, then the privacy loss distribution of the end-to-end procedure is also Gaussian; this is because composition simply adds/convolves the privacy losses. Given a lower bound for a single step of DP-SGD, $\epsilon_{\text{emp}}^{\text{lower}}(\delta)$, we find a Gaussian noise scale σ that corresponds to this (ϵ, δ) -DP guarantee. Then we compose the corresponding Gaussian DP guarantees to obtain our final estimate.

In DP-SGD, there is also subsampling. That is, we must account for the randomness of the batch selection. In this case, the privacy is not tightly characterized by Gaussian DP.¹ However, it can be characterized by a more general f -DP guarantee and then we can use PLD to compose over the number of update steps. In Section 6, we show that this method of conversion gives empirical estimates for ϵ that are close to the end-to-end theoretical ϵ value. We note that a similar idea has been explored by Maddock *et al.* [18].

¹ Although it is not tightly characterized by Gaussian DP, a subsampled Gaussian can be approximated by Gaussian DP. In practice, this approximation yields conservative estimates of the final (ϵ, δ) -DP guarantee. Hence this would also be an acceptable auditing methodology.

6 Experiments

We now evaluate the performance of our proposed auditing technique. We first demonstrate that auditing with f -DP gives a tight bound on privacy leakage. After this, we show that tight auditing can be used for a multitude of purposes, such as investigating if certain choices of training hyperparameters lead to more or less privacy leakage, and debugging implementations of DP-SGD.

6.1 Experiment Setup

We experiment with two commonly used datasets in the privacy literature: CIFAR-10 (with Wide ResNet (WRN-16) [29] and ConvNet architectures) and Purchase. In addition we also evaluate our experiment on a randomly initialized dataset. In Appendix B we describe the hyperparameters and details used to train models. Unless otherwise stated, all lower bounds are given with a 95% confidence (Clopper-Pearson as in Nasr et al. [21]) / credible interval (Zanella-Béguelin et al. [30]), and we audit in the *White-box access with Gradient Canaries* threat model using Algorithm 2.

Terminology: We describe below the approaches we use to compute lower bounds; our work introduces the f -DP strategies. f -DP (CP): using the trade-off function of the privacy mechanism with Clopper-Pearson. When we do not have the exact trade-off function (e.g, if there are multiple composition of sub-sampled Gaussian mechanisms) we use approximated trade-off function of the privacy mechanism from PLD accounting and Algorithm 4. (ϵ, δ) -DP (CP) [12, 21]: using the (ϵ, δ) -DP (Equation (4)) trade-off function with Clopper-Pearson. f -DP (ZB) and (ϵ, δ) -DP (ZB) are similar to f -DP (CP) and (ϵ, δ) -DP (CP), however, we use the Bayesian estimation approach (Section 4.2) to compute lower bounds instead of Clopper-Pearson. (ϵ, δ) -DP (ZB) is equivalent to the approach used by Zanella-Béguelin et al. [30]. ϵ -DP (Katz) [17] audits ϵ -DP with the Katz log confidence interval.

6.2 (Almost) Tight Auditing of DP-SGD For Natural Datasets Using f -DP

Nasr et al. [21] showed that DP-SGD is tight with worst-case training sets; however, they observed a noticeable gap between the empirically estimated lower bound and theoretic upper bound for ϵ when they replace these worst-case datasets with datasets commonly used for DP-SGD benchmarking, even when the adversary has white-box access to the model and can insert canary gra-

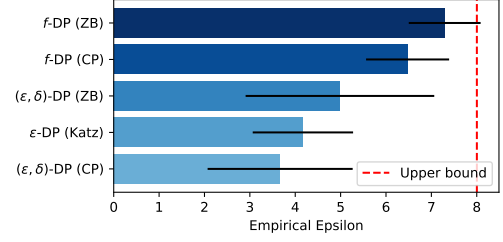


Figure 6: Auditing with f -DP provides the strongest lower bounds with theoretical upper bound $\epsilon = 8$ on CIFAR-10.

Table 1: Comparison of the empirical lower bounds on epsilon with 95% (confidence or credible interval), where the adversary has access to every intermediate model and the adversary can insert a canary gradient vector (white-box setting). We include results on a *Random* dataset—random pixels and labels—of the same cardinality as CIFAR-10.

Lower Bounding	Theoretical ϵ	CIFAR-10 WRN-16	CIFAR-10 ConvNet	Purchase	Random WRN-16
f -DP (CP)	1	0.75	0.77	0.78	0.74
	4	3.40	3.34	3.54	3.14
	8	5.80	6.12	6.40	7.14
	16	11.14	12.08	12.42	13.14
f -DP (ZB)	1	0.95	0.94	0.89	0.90
	4	3.73	3.80	3.60	3.52
	8	7.09	7.12	6.94	7.12
	16	13.95	13.80	13.80	15.14
(ϵ, δ) -DP (CP)	1	0.41	0.45	0.36	0.35
	4	1.37	1.80	1.65	1.14
	8	3.63	3.85	3.25	4.09
	16	5.25	6.22	6.34	6.96
(ϵ, δ) -DP (ZB)	1	0.62	0.62	0.57	0.61
	4	2.65	2.69	2.45	2.75
	8	5.07	5.15	4.65	5.09
	16	5.25	6.22	6.34	6.96
ϵ -DP (Katz)	1	0.49	0.51	0.46	0.41
	4	1.65	1.95	2.05	2.14
	8	4.17	3.95	4.24	4.15
	16	7.52	7.63	7.69	8.01

dients (e.g., they achieved an empirical ϵ lower bound of < 1 with a theoretical ϵ of 8 on CIFAR-10).

We first demonstrate that by auditing with f -DP, we can now compute strong lower bounds with the standard CIFAR-10 training set, where we train and audit a model with 79% test accuracy at $(\epsilon = 8, \delta = 10^{-5})$ -DP. We evaluate and compare our auditing technique against the state-of-the-art auditing methods of Zanella-Béguelin et al. [30] and Lu et al. [17]. Results are given in Figure 6, where we report the average lower bound found over ten independent executions of the experiment along with standard deviation, and the theoretical upper bound for ϵ given by the privacy accountant. Regardless of if we use Zanella-Béguelin et al.’s method for finding credible intervals, or use the Clopper-Pearson confidence interval, the main gain in estimating ϵ comes from auditing with f -DP. Auditing with f -DP is almost tight, while the strongest upper bound from prior work is ~ 5 .

Our method of auditing with f -DP gives a tight analysis for privacy leakage for both small and large ϵ and across different datasets (CIFAR-10, Purchase, and a *Random* dataset—random pixels and labels—of the same cardinality as CIFAR-10). The results, reported in Table 1, show that our approach *does not require the use of a worst-case dataset to achieve tight estimation of the privacy parameters*. Lu et al. [17] hypothesize that privacy is dataset dependent even in a white-box setting, however, our experiments contradict this hypothesis. Given that our results show that tight lower bounds are largely independent of the choice of dataset if the adversary audits in a white-box threat model with canary gradients, our remaining experiments will focus primarily on the CIFAR-10 dataset unless stated otherwise.

We next demonstrate that auditing with f -DP can be useful for detecting implementations of DP-SGD that violate the purported upper bound for ϵ . As discussed previously, we will concentrate on violations that are not directly caused by sub-sampling, and so our experiments will audit the Gaussian mechanism without composition or sub-sampling, for which we will use the exact trade-off function (GDP instead of the PLD approximation detailed in Section 4.1).

6.3 Auditing and Debugging DP-SGD Implementations

Implementing DP-SGD correctly is notoriously difficult. Auditing can help identify issues of correctness, as demonstrated by Tramèr et al. [24] who used black-box auditing to show the DP-SGD implementation proposed by Stevens et al. [23] was incorrect and reported a much lower value of ϵ than its true privacy leakage.

We investigate how easily our method of auditing with f -DP can detect incorrect implementations of DP-SGD compared to prior work on CIFAR-10. The upper bound for ϵ claimed by each DP-SGD implementation throughout the following experiments is 1.27. For all experiments in this section we audit a step of DP-SGD, that is, we do not convert our lower bounds into a guarantee of the ϵ reported after composing across all training steps with PLD. We do this because even if we were to report a lower bound on the final value of ϵ (via finding a lower bound for a step of DP-SGD and composing with the (almost) lossless PLD), there will be bugs that cannot be captured by this auditing method. For example, a bug that is caused by implementing a biased sub-sampling method from the training dataset will likely not be captured by our audit.

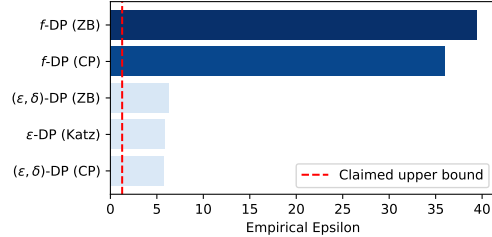


Figure 7: Clipping after gradient averaging bug. We plot the lower bound for ϵ we can find with each auditing technique when the implementation clips the gradient after averaging in a batch, and so is not (ϵ, δ) -DP with the claimed $\epsilon = 1.27$. All methods are able to detect a violation but only f -DP auditing can show the implementation is completely broken, as we can show $\epsilon > 35$.

Violation 1: Clipping *after* gradient averaging. In DP-SGD, individual gradients must be clipped to a maximum norm C before aggregation. If the order of these operations is reversed, clipping after aggregation, then the model will not be (ϵ, δ) -DP. In Figure 7, we see all auditing methods are able to identify a violation as the lower bound found is much larger than the reported upper bound. However, one may still incorrectly assume that the implementation retains some privacy if we do not audit with f -DP, as the best lower bound we can find is < 10 . By auditing with f -DP, it becomes clear that the implementation is completely broken.

Violation 2: Biased noise sampling. At every step of DP-SGD, random Gaussian noise must be added to gradients. If the noise is not randomly sampled then the model will not be (ϵ, δ) -DP. In practice, we generate a Gaussian noise sample by seeding a random number generator. We train a model where the seed can only take on 100 different possible values, meaning there are only 100 different possible Gaussian noise vectors. It may seem that this is a rather contrived example of a DP-SGD bug, but a similar error appeared in the JAX canonical example of how to implement DP-SGD, where a random seed was re-used when adding noise to different sets of model parameters [1].

Results are shown in Figure 8. We can detect violations when auditing with f -DP; the auditing method introduced by Lu et al. [17] also successfully detects the bug. However, auditing with (ϵ, δ) -DP directly, either using Clopper-Pearson or the method proposed by Zanella-Béguélin et al. was not able to identify a violation of the claimed upper bound.

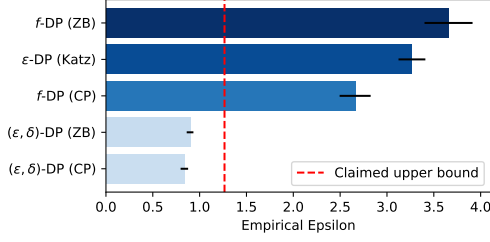


Figure 8: Biased noise bug. The Gaussian noise used to privatize gradients in DP-SGD is not sampled randomly. Both f -DP and Lu et al. detect this implementation issue while auditing with (ϵ, δ) -DP fails to detect the issue.

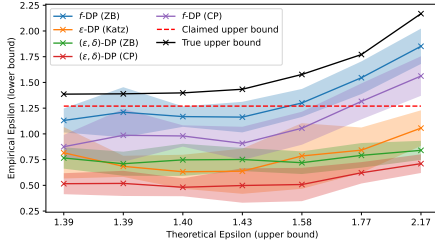


Figure 9: Incorrect noise scale bug. We measure how quickly each auditing method can detect a violation of the purported upper bound of $\epsilon = 1.27$ when the scale of noise we add to gradients is incorrectly set.

Violation 3: Noise scale is too small. The value of ϵ is inversely proportional to the scale of noise we add to gradients. As we decrease the scale of noise, ϵ increases. The third bug we investigate is when the noise scale we add is unexpectedly smaller than the target scale we set. This bug often arises because the sensitivity (clipping value) needs to be calibrated to the batch size in order to compute the correct noise scale, and this is easy to get wrong (c.f. Tramèr et al. [24]). For example, in settings where gradient computations are distributed across multiple machines, we could incorrectly add noise to the average gradient found on each machine, and then aggregate.

We train models with decreasing scales of noise, implying larger values of ϵ than the claimed upper bound. Results are shown in Figure 9, where we find auditing with f -DP closely follows the true upper bound, meaning we can detect a violation to reported $\epsilon = 1.27$ when the true value is $\epsilon = 1.57$. Auditing with (ϵ, δ) -DP directly, either using Clopper-Pearson, Lu et al. [17], or the method proposed by Zanella-Béguélin et al. does not successfully identify a violation of the claimed upper bound, even when the true upper bound for ϵ is as large as 2.17.

6.4 Investigating Privacy Leakage with Tight Auditing

In this section, we expand our analysis to examine the impact of various settings and parameters on privacy leakage.

Is there a difference between auditing in gradient and input space? Auditing with gradient canaries can be useful in specific contexts, such as in federated learning or when debugging a model. However, in most cases, practitioners are more concerned with understanding the effect of a single input space example on the model. This is because this setting more closely measures the privacy leakage that could be experienced by a worst-case training example.

To evaluate how privacy leakage could change by removing the ability to insert a canary gradient, we run experiments in the *White-box access with Input Space Canaries* threat model, as described in Section 3. In particular, we use Algorithm 3 to create the canaries. Note that for this experiment we only use the first 250 iterations of DP-SGD to collect observations and estimate the per-step ϵ lower bound. We found that, in this threat model, the first few hundred iterations of DP-SGD leaks more privacy than the entire training run; in other words, the lower bound we compute using the first 250 steps is larger than the lower bound we compute over the entire training run (2,500 update steps). In general, we find that auditing in the input space becomes weaker if the observations are collected from updates towards the end of training. We discuss this further in Appendix D.1.

As shown in Figure 10, even if the attacker can only insert canary samples (rather than gradients), we can compute tight bounds for $\epsilon < 10$. We also observe that the choice of the model architecture has an impact on auditing in input space. Specifically, we can get an (almost) tight lower bound when using Wide Resnet architecture regardless of the datasets. Moreover, when we compare the CIFAR-10 dataset results between Wide Resnet and ConvNet models, we see a large gap which further emphasizes the significance of the model architecture on privacy leakage.

Does clipping alone help? It has been conjectured that clipping individual gradients can provide some privacy even without adding noise [5]. Technically, these models are not differentially private but we can measure the privacy that clipping provides by computing an ϵ lower bound. Of course, it doesn't make sense to audit clipping alone in a white-box access threat model; no noise is

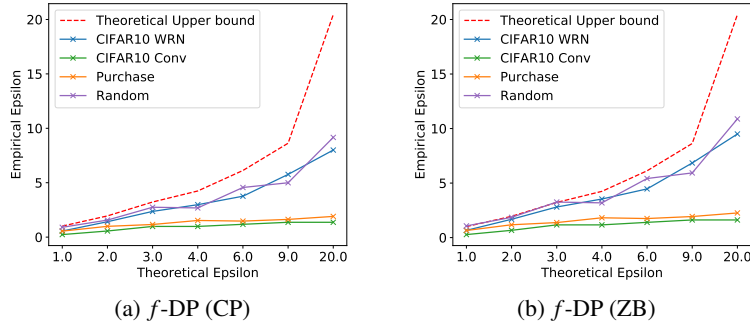


Figure 10: Comparison of the empirical lower bounds on epsilon with 95% (confidence or credible interval), where the adversary has access to every intermediate model and the adversary can insert a canary in input space (white-box setting).

Table 2: Comparison of the empirical lower bounds on ϵ (using f -DP (ZB)) in the black-box threat model where the adversary inserts a canary point at the beginning of training, and only gradient clipping is applied on CIFAR-10.

Clipping norm C	ϵ lower bound
0.1	23.4
1.0	26.8
10.0	41.0
Maximum ϵ lower bound at this number of observations	44.0

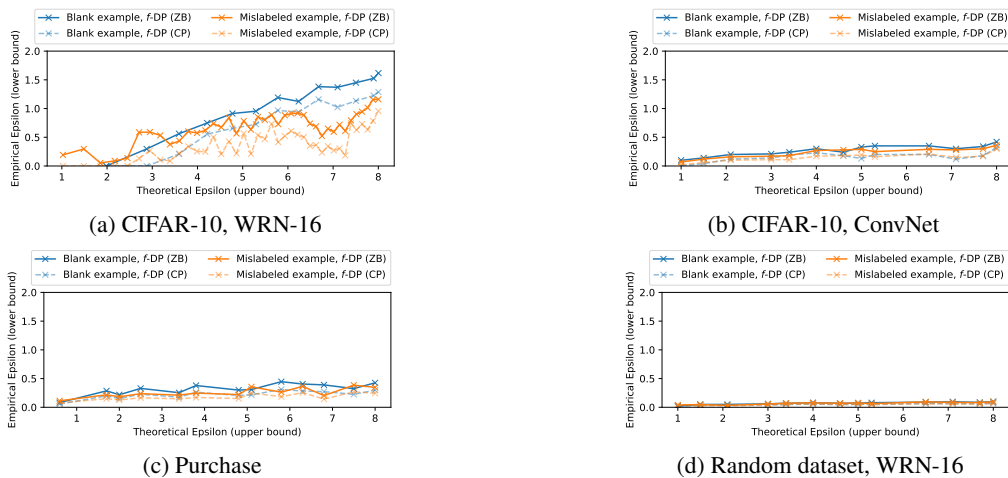


Figure 11: Auditing the black-box threat model with f -DP. We train 1,000 models with and without the canary sample that we insert at the beginning of training. We either use a blank or mislabeled image as the canary input.

added and so the dot-product value we compute to find type I and type II error rates (as described in Section 5.1 and Section 5.2) will not be masked by any noise. Instead, we audit in the black-box threat model by creating a canary point, inserting it into the training set, and then training a CIFAR-10 WRN-16 model. We then measure the loss after training on the canary image. We do this 1,000 times when the canary image was included in training and when it wasn't, and record the loss of the canary image in each case. We experimented with a range of different canary inputs, but found that blank (white) and mislabeled images produced the strongest lower bounds for ϵ . In other words, these two canary types had losses that were easily separable depending on if they were included in training or not.

In total, we train 2,000 models: 1,000 when the canary input was in training and 1,000 when the canary wasn't included. Our results are shown in Table 2, where we see

that clipping alone provides no privacy. The lower bounds for a clipping norm of 10 are close to the maximum possible lower bound at this number of observations.

How much privacy is leaked in a black-box threat model? To measure the impact of switching to a black-box threat model on privacy leakage estimation, we follow a similar auditing procedure as set out by Nasr et al. [21]. We select a canary example, then we train 1,000 models with the canary point (+ training set), and 1,000 models with only the training set (canary excluded). We then measure the $\log(\frac{p}{1-p})$ for each model, where p is the probability of the canary point with respect to its label. We take the distribution of $\log(\frac{p}{1-p})$ when the canary point was and wasn't in the training set, and compute ϵ lower bounds using our f -DP method.

Results are shown in Fig. 11, where all models are

trained up to $\epsilon = 8$. On CIFAR-10 with a Wide ResNet architecture we are able to find a lower bound of ~ 1.6 using f -DP (ZB) auditing. However, it is difficult to separate the effects of the black-box threat model from the effect that sub-sampling has on privacy leakage estimation, as we saw in Figure 5, both GDP and PLD tend to overestimate the observed trade-off between type I and type II errors in the sub-sampled Gaussian mechanism (with composition over multiple iterations). Additionally, the auditing results on other datasets/architectures are significantly lower compared to those obtained with the Wide ResNet architecture on the CIFAR-10 dataset. Specifically, when examining the random dataset, no non-trivial lower bounds for auditing can be achieved. One of the key factors that distinguishes these experiments is the final accuracy the model is able to attain. At $\epsilon = 8$, the Wide ResNet architecture for CIFAR-10 is able to achieve a test dataset accuracy of greater than 70%, compared to less than 50% for the ConvNet architecture and less than 50% for the Purchase dataset. Our results suggest that there remains a gap between the theoretical privacy upper bound and the empirical lower bound that can be achieved in a black-box setting, but the size of this gap is highly dependent on the dataset and model architecture.

7 Conclusion

As differentially private machine learning becomes more popular and fewer expert users begin to implement such methods, the possibility of bugs and implementation errors will increase. We provide a simple auditing technique that can achieve tight estimates of privacy leakage on standard ML benchmark datasets. Our method can be easily integrated with privacy preserving libraries (TF-privacy, Opacus, JAX privacy) to give online estimation of the private mechanism parameters and provide an empirical test for the assumed privacy budget, and only increases the computational overhead by a factor of two.

References

- [1] fix prng key reuse in differential privacy example by mattjj · Pull Request #3646 · google/jax — github.com. <https://github.com/google/jax/pull/3646>. [Accessed 28-Jan-2023].
- [2] ABADI, M., CHU, A., GOODFELLOW, I., McMAHAN, H. B., MIRONOV, I., TALWAR, K., AND ZHANG, L. Deep learning with differential privacy. In *Proceedings of the 2016 ACM SIGSAC conference on computer and communications security* (2016), pp. 308–318.
- [3] ALTSCHULER, J. M., AND TALWAR, K. Privacy of noisy stochastic gradient descent: More iterations without more privacy loss. *arXiv preprint arXiv:2205.13710* (2022).
- [4] BALLE, B., CHERUBIN, G., AND HAYES, J. Reconstructing training data with informed adversaries. *arXiv preprint arXiv:2201.04845* (2022).
- [5] CARLINI, N., CHIEN, S., NASR, M., SONG, S., TERZIS, A., AND TRAMER, F. Membership inference attacks from first principles. In *2022 IEEE Symposium on Security and Privacy (SP)* (2022), IEEE, pp. 1897–1914.
- [6] CARLINI, N., IPPOLITO, D., JAGIELSKI, M., LEE, K., TRAMER, F., AND ZHANG, C. Quantifying memorization across neural language models. *arXiv preprint arXiv:2202.07646* (2022).
- [7] CARLINI, N., LIU, C., ERLINGSSON, Ú., KOS, J., AND SONG, D. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *28th USENIX Security Symposium (USENIX Security 19)* (2019), pp. 267–284.
- [8] DE, S., BERRADA, L., HAYES, J., SMITH, S. L., AND BALLE, B. Unlocking high-accuracy differentially private image classification through scale. *arXiv preprint arXiv:2204.13650* (2022).
- [9] DONG, J., ROTH, A., AND SU, W. J. Gaussian differential privacy. *arXiv preprint arXiv:1905.02383* (2019).
- [10] DWORK, C., MCSHERRY, F., NISSIM, K., AND SMITH, A. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference* (2006), Springer, pp. 265–284.
- [11] JAGIELSKI, M., THAKKAR, O., TRAMER, F., IPPOLITO, D., LEE, K., CARLINI, N., WALLACE, E., SONG, S., THAKURTA, A., PAPERNOT, N., ET AL. Measuring forgetting of memorized training examples. *arXiv preprint arXiv:2207.00099* (2022).
- [12] JAGIELSKI, M., ULLMAN, J., AND OPREA, A. Auditing differentially private machine learning: How private is private sgd? *Advances in Neural Information Processing Systems 33* (2020), 22205–22216.

- [13] JAYARAMAN, B., AND EVANS, D. Evaluating differentially private machine learning in practice. In *USENIX Security Symposium* (2019).
- [14] KAIROUZ, P., OH, S., AND VISWANATH, P. The composition theorem for differential privacy. In *International conference on machine learning* (2015), PMLR, pp. 1376–1385.
- [15] KATZ, D., BAPTISTA, J., AZEN, S., AND PIKE, M. Obtaining confidence intervals for the risk ratio in cohort studies. *Biometrics* (1978), 469–474.
- [16] KOSKELA, A., JÄLKÖ, J., AND HONKELA, A. Computing tight differential privacy guarantees using fft. In *International Conference on Artificial Intelligence and Statistics* (2020), PMLR, pp. 2560–2569.
- [17] LU, F., MUNOZ, J., FUCHS, M., LEBLOND, T., ZARESKY-WILLIAMS, E., RAFF, E., FERRARO, F., AND TESTA, B. A general framework for auditing differentially private machine learning. *arXiv preprint arXiv:2210.08643* (2022).
- [18] MADDOCK, S., SABLAYROLLES, A., AND STOCK, P. Canife: Crafting canaries for empirical privacy measurement in federated learning. *arXiv preprint arXiv:2210.02912* (2022).
- [19] MIRONOV, I. Rényi differential privacy. In *2017 IEEE 30th computer security foundations symposium (CSF)* (2017), IEEE, pp. 263–275.
- [20] NASR, M., SHOKRI, R., AND HOUMANSADR, A. Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning. In *2019 IEEE symposium on security and privacy (SP)* (2019), IEEE, pp. 739–753.
- [21] NASR, M., SONGI, S., THAKURTA, A., PAPERNOT, N., AND CARLINI, N. Adversary instantiation: Lower bounds for differentially private machine learning. In *2021 IEEE Symposium on security and privacy (SP)* (2021), IEEE, pp. 866–882.
- [22] SHOKRI, R., STRONATI, M., SONG, C., AND SHMATIKOV, V. Membership inference attacks against machine learning models. In *2017 IEEE symposium on security and privacy (SP)* (2017), IEEE, pp. 3–18.
- [23] STEVENS, T., NGONG, I. C., DARAIS, D., HIRSCH, C., SLATER, D., AND NEAR, J. P. Backpropagation clipping for deep learning with differential privacy. *arXiv preprint arXiv:2202.05089* (2022).
- [24] TRAMÈR, F., TERZIS, A., STEINKE, T., SONG, S., JAGIELSKI, M., AND CARLINI, N. Debugging differential privacy: A case study for privacy auditing. *arXiv preprint arXiv:2202.12219* (2022).
- [25] TRAMER, F., TERZIS, A., STEINKE, T., SONG, S., JAGIELSKI, M., AND CARLINI, N. Debugging differential privacy: A case study for privacy auditing. *arXiv preprint arXiv:2202.12219* (2022).
- [26] WASSERMAN, L., AND ZHOU, S. A statistical framework for differential privacy. *Journal of the American Statistical Association* 105, 489 (2010), 375–389.
- [27] YE, J., AND SHOKRI, R. Differentially private learning needs hidden state (or much faster convergence). *arXiv preprint arXiv:2203.05363* (2022).
- [28] YEOM, S., GIACOMELLI, I., FREDRIKSON, M., AND JHA, S. Privacy risk in machine learning: Analyzing the connection to overfitting. In *2018 IEEE 31st computer security foundations symposium (CSF)* (2018), IEEE, pp. 268–282.
- [29] ZAGORUYKO, S., AND KOMODAKIS, N. Wide residual networks. In *British Machine Vision Conference 2016* (2016), British Machine Vision Association.
- [30] ZANELLA-BÉGUELIN, S., WUTSCHITZ, L., TOPLE, S., SALEM, A., RÜHLE, V., PAVERD, A., NASERI, M., AND KÖPF, B. Bayesian estimation of differential privacy. *arXiv preprint arXiv:2206.05199* (2022).

A Using PLD to approximate the trade off function

PLD does not have a closed-form trade-off function, but we can evaluate empirically a lower bound for a given FPR. We approximate the trade-off function using Algorithm 4, which will give us a looser bound for ϵ , however, the difference is in the order of 10^{-1} . Figure 12 compares the approximation approach in Algorithm 4 to the optimal trade-off function, as we can see, even with 10 approximations we can get a good estimation of the trade-off function. In Section 6, we will present experiments that provide lower bounds for a single step of DP-SGD

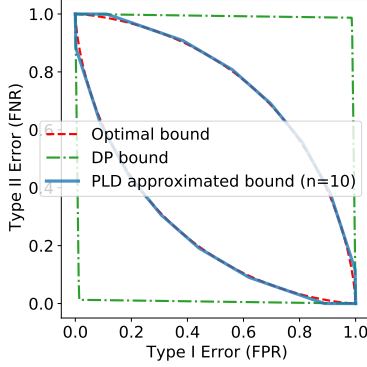


Figure 12: Comparison of the trade-off functions when using an approximation of PLD compared to the optimal curve.

and multiple steps, they will use the GDP and PLD formulations, respectively.

Algorithm 4 Approximating a lower bound on trade-off function using PLD

Args: $f_{\mathcal{M}}$ privacy analysis function (outputs ϵ for a given δ), n number of approximation lines, δ target delta in privacy analysis
 $\Delta \leftarrow n$ linearly spaced points between $[\delta, 1 - \delta]$
for $\delta' \in \Delta$ **do**
 $\hat{\epsilon} \leftarrow f_{\mathcal{M}}(\delta')$
 $l_{\delta'}(x) := \max(0, 1 - \delta' - (xe^{\hat{\epsilon}}), e^{-\hat{\epsilon}(1-\delta'-x)})$
end for
 $l(x) := \min_{\delta' \in \Delta} l_{\delta'}(x)$
return l

B Experimental Setup

On CIFAR-10, we train a Wide ResNet (WRN-16) model [29] for 2,500 steps with a batch size of 4,096. This reaches approximately 79% test accuracy at $(\epsilon = 8, \delta = 10^{-5})$ -DP, which is on a par with state-of-the-art [8]. We also use a ConvNet architecture with batch size set to 256 and 20,000 steps, which achieves approximately 60% test accuracy at $(\epsilon = 8, \delta = 10^{-5})$ -DP. To evaluate the effect of the choice of dataset on privacy leakage, we also experiment with a randomly initialized dataset with the same dimensionality and cardinality as the CIFAR-10 dataset but with pixel values and labels randomly assigned with Wide ResNet architecture. For the Purchase dataset we use a three layer fully connected network.

For each dataset in the white-box threat model, we

train two models, one without canaries and one where the canary sampling rate is q_c . Unless otherwise stated, we set $q_c = 1$. Of course, we could increase the number of training steps to create a larger number of observations, but we found our method is strong enough to produce tight bounds for large ϵ with as little as a few hundred observations (as shown in Figure 3).

C Effect Of Auditing Choices

C.1 How to choose a decision threshold given a set of observations

Our gradient-based auditing method will generate data from two Gaussian distributions: $O = \mathcal{N}(0, \sigma^2 I)$ and $O' = \mathcal{N}(1, \sigma^2 I)$. We must choose a threshold from which we predict if observations are from O or O' . This threshold should be chosen on another set of held-out set of observations in order for our lower bound to be valid. That is, we cannot choose the threshold that is dependent on the observations from which we compute the lower bound. We discuss how this threshold should be chosen if one does not have access to a hold-out dataset of observations (e.g. because generating them is prohibitively expensive) but we want our bounds to remain valid.

As a reminder, Nasr et al. [21] compute the following lower bound:

$$\epsilon \geq \max \left[\ln \left(\frac{1 - \delta - \alpha}{\beta} \right), \ln \left(\frac{1 - \delta - \beta}{\alpha} \right), 0 \right] \quad (15)$$

where α and β are the false positive and false negative rate of the audit. The false negative rate (β) is given by $\Phi(\frac{z-c}{\sigma})$ and the false positive rate (α) is given by $1 - \Phi(\frac{z}{\sigma})$. Because the ϵ lower bound is symmetric around the decision threshold $\frac{c}{2}$, we can consider only positive decision thresholds $z > \frac{c}{2}$, where $c = 1$. For $z > \frac{c}{2}$ the false positive rate α decreases at a faster rate than the false negative rate β , and so $\max \left[\ln \left(\frac{1 - \delta - \alpha}{\beta} \right), \ln \left(\frac{1 - \delta - \beta}{\alpha} \right), 0 \right] = \max \left[\ln \left(\frac{1 - \delta - \beta}{\alpha} \right), 0 \right]$.

Plugging in $\beta = \Phi(\frac{z-c}{\sigma})$ and $\alpha = 1 - \Phi(\frac{z}{\sigma})$ gives:

$$\epsilon \geq \max \left[\ln \left(\frac{1 - \delta - \Phi(\frac{z-c}{\sigma})}{1 - \Phi(\frac{z}{\sigma})} \right), 0 \right] \quad (16)$$

Since $\ln(\cdot)$ is monotonically increasing we can find the maximum lower bound for ϵ by upper bounding:

$$h(z) = \frac{1 - \delta - \Phi(\frac{z-c}{\sigma})}{1 - \Phi(\frac{z}{\sigma})} \quad (17)$$

Setting $h'(z) = 0$ gives:

$$\delta = \Phi\left(\frac{c-z}{\sigma}\right) - \frac{\phi\left(\frac{c-z}{\sigma}\right)}{\phi\left(\frac{-z}{\sigma}\right)} \Phi\left(\frac{-z}{\sigma}\right) \quad (18)$$

where $\Phi'(\cdot) = \phi(\cdot)$. Let w denote the value of z that satisfies Eq. (18). Then the maximum ϵ lower bound is given by:

$$\ln\left(\frac{1 - (\Phi(\frac{c-w}{\sigma}) - \frac{\phi(\frac{c-w}{\sigma})}{\phi(\frac{-w}{\sigma})} \Phi(\frac{-w}{\sigma})) - \Phi(\frac{w-c}{\sigma})}{1 - \Phi(\frac{w}{\sigma})}\right) \quad (19)$$

$$= \ln\left(\frac{\phi(\frac{c-w}{\sigma})}{\phi(\frac{-w}{\sigma})}\right) \quad (20)$$

$$= \frac{1}{2\sigma^2}(w^2 - (w-c)^2) \quad (21)$$

$$= \frac{c}{2\sigma^2}(2w-c) \quad (22)$$

This is positive because we know $w > \frac{c}{2}$. Importantly, the maximum lower bound is only achieved if we choose a decision threshold w satisfying Eq. (18), which only has a single solution.

Our audit method relies on estimating μ in μ -GDP and then converting to (ϵ, δ) -DP. Note that the Gaussian mechanism that adds $\mathcal{N}(0, \sigma^2 I)$ noise to the statistic θ , gives μ -GDP if $\sigma = \frac{\epsilon}{\mu}$. This means estimating μ is equivalent to estimating σ given a decision rule and observations from O and O' .

For μ -GDP we have

$$\beta_\alpha := \inf\{\beta_z : \alpha_z \leq \alpha\} \quad (23)$$

$$= \Phi(\Phi^{-1}(1-\alpha) - \mu) \quad (24)$$

where z is a decision threshold on the real line. For any z we have:

$$\Phi^{-1}(1-\alpha) - \Phi^{-1}(\beta) = \Phi^{-1}(1 - 1 + \Phi(\frac{z}{\sigma})) - \Phi^{-1}(1 - \Phi(\frac{z-1}{\sigma})) \quad (25)$$

$$= \frac{z}{\sigma} - \Phi^{-1}(\Phi(\frac{1-z}{\sigma})) \quad (26)$$

$$= \frac{1}{\sigma} \quad (27)$$

That is, given two Gaussian distributions, the lower bound derived from GDP is independent of the threshold. Given a sufficient number of observations from O and O' , our lower bounds found through GDP will be

approximately equal, while we must use the threshold w to maximize the lower bound found through (ϵ, δ) -DP auditing.

We empirically show that difference in bounds between our GDP method and (ϵ, δ) -DP with Clopper-Pearson for different thresholds at a sample size of observations equal to $5K$ in Fig. 4 with $\sigma = 1$. As expected, different decision thresholds lead to approximately the same lower bound found through GDP auditing, while the lower bounds drastically change for (ϵ, δ) -DP auditing with Clopper-Pearson. Furthermore, the optimal threshold for (ϵ, δ) -DP auditing (given by Eq. (18)) with Clopper-Pearson gives a lower bound of zero since there aren't a sufficient number of observations to cover this region of the observation space. When auditing with (ϵ, δ) -DP with Clopper-Pearson, we show that the optimal threshold given by Eq. (18) decision threshold can achieve tight lower bounds given enough samples from O and O' in Fig. 13, while other thresholds cannot, again, as predicted by our theory.

In sum, we can achieve tight lower bounds with GDP, and with as little as $5K$ samples we can expect any decision threshold to give strong lower bounds, removing our need for hold-out observations to find the optimal threshold. Using the (ϵ, δ) -DP with Clopper-Pearson lower bound requires us to either (1) use an extremely large number of samples and then use the optimal decision threshold – this will be prohibitively expensive as this would require $> 100M$ observations, or (2), use a random / guess threshold which will saturate far below the upper bound for any number of observations. From Fig. 13, the maximum lower bound found through Clopper-Pearson requires approximately $250M$ observations to reach the same lower bound found through auditing with Clopper-Pearson and GDP. In other words, we would need approximately $5000x$ more observations.

C.2 Choosing a Canary Gradient

We first investigate how the canary gradient affects the estimated privacy bound. We construct the canary gradient in three ways:

1. Dirac canary: All gradient values are zero except at a single index.
2. Constant canary: All gradient values have the same value.
3. Random canary: Gradient sampled from a Gaussian.

In each setting, gradient values are re-scaled such that the canary gradient has a maximum norm equal to the clipping norm. We also measure if there is a difference

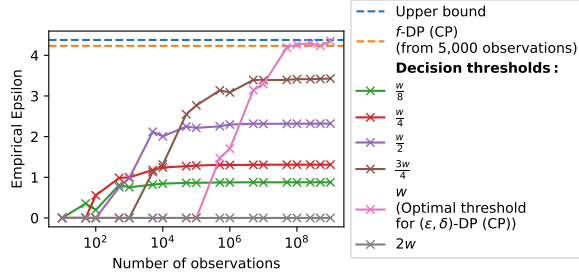


Figure 13: We compare how the number of observations affect the lower bound found through (ϵ, δ) -DP with Clopper-Pearson using different decision thresholds. The optimal threshold as set out by Eq. (18) only reaches the upper bound when the number of observations is $>100M$. The lower bounds found with other decision thresholds saturate at smaller values than the upper bound.

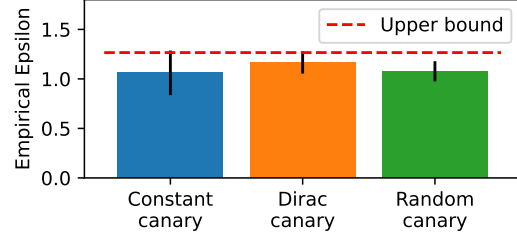
between using the same canary gradient at each iteration of DP-SGD or creating a new canary gradient. For example, in the Dirac canary we would randomly sample a new index to set to the clipping norm at each update. Results are shown in Fig. 14; using a Dirac canary that is reset at each update performs best.

C.3 Choosing a Canary Input in The White-box Setting

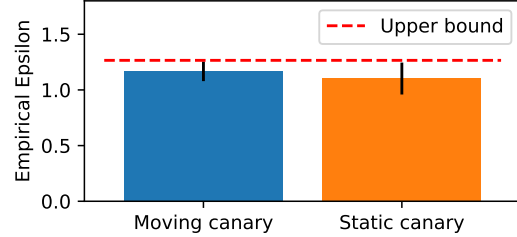
We evaluate how different types of input canaries can affect auditing in a white-box setting. We construct the canaries in four different ways:

1. **Mislabeled example:** We select a random example from the test dataset of the model and we select a random label (that is not equal to the original label).
2. **Blank example:** We craft an input where all dimensions of the input are equal to zero.
3. **Adversarial example:** We apply Projected Gradient Descent (PGD) to generate adversarial example on a random example from the test dataset.
4. **Crafted example:** We use Algorithm 3 to generate an input example.

As we saw from our gradient experiment using the dot product between the privatized gradient and the canary gradient is a sufficient metric for auditing DP-SGD. Therefore we use the same idea in our crafting algorithm (Algorithm 3) in input space. We look for a canary such that its gradient is orthogonal to other gradients in the training batch. However, we cannot use the example in the training batch directly to craft such an example as it will violate the DP-assumptions (the adversary cannot



(a) We compare three different ways to create a canary gradient. The Dirac canary gradient, with zeros everywhere except for a single position which has a value set to the clipping norm, slightly outperforms other approaches.



(b) At each iteration we can either insert the same (static) canary gradient, or compute a new canary gradient (moving). There is little difference between these two approaches in terms of the ϵ lower bound we can find.

Figure 14: How design decisions for the canary gradient change the lower bound we compute for ϵ with f -DP.

have access to non-noisy gradients). Therefore, we assume the adversary has access to an example from the same distribution as the training dataset and uses that data to estimate the gradient of the model on the training example. Then, it crafts an example such that its gradient is orthogonal to the estimated gradient. Thus, creating a gradient that is significantly different from other examples in the batch and its presence can be detected.

Figure 15 compares the effectiveness of different input canaries in the white-box setting. As can be seen, using our canary crafting approach we can achieve significantly tighter bounds on DP-SGD compared to other canary crafting strategies. Unfortunately, it is not trivial to extend either the adversarial example or our crafting approach to the black-box setting and therefore we do not use them in the black-box experiments.

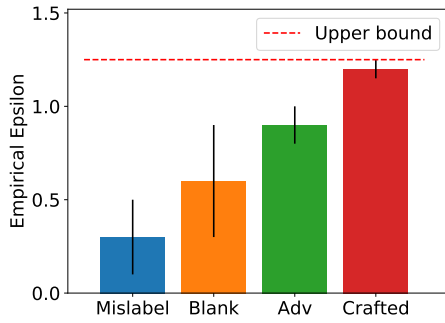


Figure 15: Comparison of the different canary crafting approaches in input space for CIFAR-10 dataset using WRN architecture.

D The effect of model related parameters

D.1 Do earlier training steps leak more information?

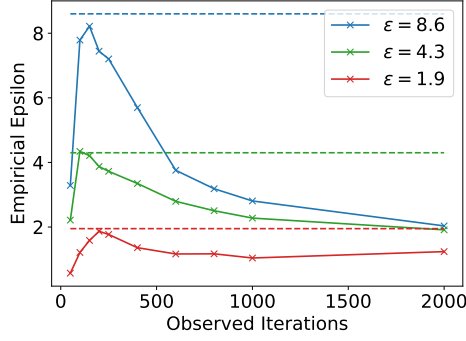
Recent work on measuring the amount of privacy leaked when training convex models with DP has shown that the true amount of leakage plateaus as the model converges [3, 27]. This means that after a certain number of steps, training for more iterations does not consume any more of the privacy budget. The analysis for this result is specific to convex models; and we cannot prove such properties for the deep models. However, when can evaluate if a similar phenomenon holds empirically. We measure the lower bounds when we only use the first n iterations of the training in the *White-box access with Input Space Canaries* threat model. Results are shown in Figure 16. We find that indeed, the first part of training does leak more information than later in training on the CIFAR-10 dataset. However, when we evaluate the random dataset we do not see the same behavior (please note that, if we only look at a very small number (<100) of the iterations we get a very loose bound on ϵ because we do not have enough observations to have a sufficiently confident estimation). Understanding why we cannot lower bound from our audit becomes looser in later iterations requires further investigation which we leave for future work. Nevertheless, the results suggest that when we limit the adversary to canaries in the input space then model architecture, underlying dataset and the how well a model has been trained all have an effect on privacy leakage.

D.2 Do larger models leak more privacy?

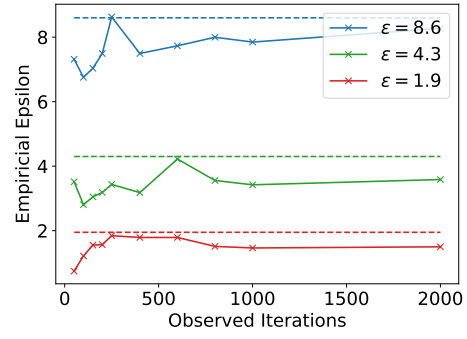
Recent work has shown the larger models have a greater capacity to memorize training data verbatim [6]. We investigate if the same trend holds when training with DP-SGD by comparing lower bounds on a WRN-16 and WRN-40 model. Results are shown in Figure 17. Interestingly, if one was to use a non f -DP auditing method, one would make an incorrect conclusion that the WRN-16 leaks more than the WRN-40. Using our f -DP auditing method, we identify that, indeed, the larger WRN-40 model leaks slightly more than the WRN-16 model.

D.3 Does augmentation multiplicity affect privacy lower bounds?

In De et al. [8], data augmentation was a key ingredient in achieving state-of-the-art results on CIFAR-10. In particular, De et al. use a data augmentation technique they term *augmentation multiplicity* (Augmult), where they augment a single example multiple times and compute the average gradient over these augmentations *before* clipping. They find that increasing the Augmult value (number of augmentations) improves performance; increasing this value does not increase the privacy cost as it does not change the sensitivity of the privatized gradient to any single example in the batch. In Fig. 18, we measure lower bounds at different Augmult values, and observe no clear trend between lower bounds and the value for Augmult.



(a) CIFAR-10 dataset with a WRN-16 model.



(b) Random dataset with a WRN-16 model.

Figure 16: Lower bounding ϵ with the input space attack (white-box setting) for different number of iterations of training and values of ϵ , using f -DP (ZB).

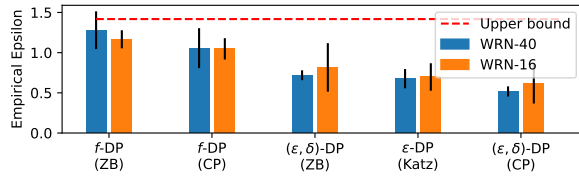


Figure 17: Comparison of how model architecture (WRN-16 and -40) affects the ϵ lower bound.

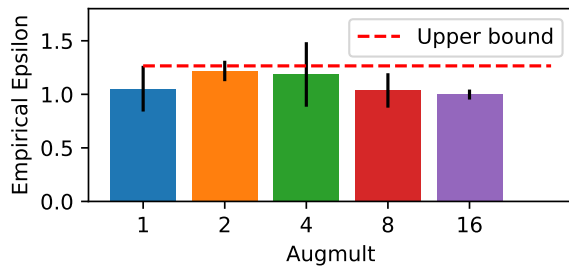


Figure 18: How the value for Augmult [8] affects the ϵ lower bound.